FPU

• IEEE 754 single-precision floating-point

32-bit allocation

Sign(S) 1 bit | Exponent(E) 8-bits | Mantissa(M) 23-bits → bits representing the fractional bits of the precision engine.

↳ used to scale the value up or down

Bias = 127

$$\text{Value} = (-1)^S \times \left(1 + \sum_{i=1}^{23} M_{23-i} \cdot 2^{-i}\right) \times 2^{(E-127)}$$

↳ to save a bit of hardware real estate IEEE754

eg -6.625_{10}

$6_{10} = 110_2$

$0.625 = 0.5 + 0.125 = 0.101_2$

Combined $= -110.101_2$

$-110.101_2 \longleftrightarrow 1.10101_2 \times 2^2$

↳ exponent = 2

$E = e + \text{Bias} = 2 + 127 = 129_{10}$

$129_{10} = 10000001$

$M = 0.10101$

$M = 10101000.0000.0000.0000.000$

↳ zero padding

$S = 1$

Combined = 1 10000001 10101000 0000 0000 0000 000

↳ help in rounding

Special Case

- Zero (0.0)
  - E = 0
  - M = 0
  - S = ±
- Infinity (∞)
  - E = 1111-1111
  - M = 0
  - Indicate division by zero or absolute overflow
- Not a Number (NaN)
  - E = 1111-1111
  - M ≠ 0
  - generated by illegal hardware process like $\sqrt{-1}$ or 0/0

Unpack A & B
(Append Hidden 1)

↓

Align [Find Exponent Difference ($d = E_a - E_b$)
shift smaller mantissa right by d bits]

↓

Add [Perform signed addition/sub on mantissa]

↓

Normal [shift result to restore '1.xxx' format
Adjust exponent accordingly]

↓

Round [Apply rounding mode using G, R, S bits]

↓

Pack [check for overflow / underflow]

Step-1 Unpacking and Breamble

The hardware extract the sign (S), exponent (E) and mantissa (M) field from both 32-bit inputs

- the FPU prepends a 1 to the MSB of each mantissa to restore the implicit leading bit (forming a 24-bit significand 1.M)
- It check if either input is zero / infinity / NaN it bypass the pipeline and route the correct special result directly to output

Step-2 Alignment (Equalizing the Exponents)

- subtract Exponent $d = E_A - E_B$
- Determine the larger Exponent : the larger exponent become the tentative exponent of the final result.
- shift right the smaller significant :

Step-3 significand Addition

- sign check if the sign of A & B are identical the hardware adds the significands
- If the signs are opposite it perform substraction (taking the 2's Complement of the negative operand)

Step-4 Normalization

- Left shift : If substraction occurred, the leading digit might be zero. the FPU uses leading point Zero Detector (LZD) to find the first 1 and shift the significant left to bring 1 back to MSB position, decreasing the exponent for every shift
- Right shift : if addition occurred the result might have carried over. the hardware shifts the right by 1 bit and increments the exponent by 1.

step 5 Rounding

The hardware uses the Guard (G), Round (R) and sticky (S) bits to round the value back to 23 bits of mantissa

- G & R the first two bits shifted out past the LSB
- S a single bit that is logically OR with all subsequent bits shifted out. If any bit past R was 1, S=1
- Using these three bits the hardware applies the selected IEEE rounding mode and adds 1 to the LSB of the significant if required. If the rounding causes a carry-out, the hardware performs one final 1-bit right shift and increments the exponent.

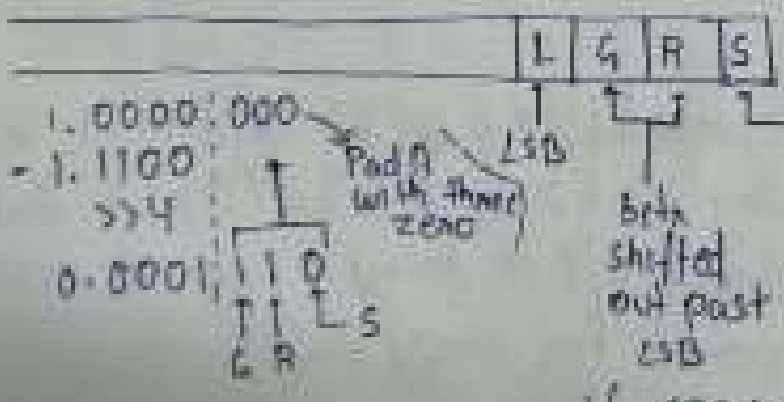
step 6 Packing and Exception Handling

the hardware strips the hidden leading 1 from the significant (leaving the 23-bit mantissa) and check errors

overflow: if the final exponent exceeds 254 the output is saturated ±∞ and the overflow exception flag is set

underflow: if the exponent is too small to represent, the number becomes subnormal or is flushed to zero

Sign, Exponent and Mantissa are packed back into the final 32-bit register.



Result 0.1110:010 S=0
<<1 G=1
1.1100:100 R=0

if GRS > 100 round up (to nearest)
GRS < 100 round down (truncate)
GRS == 100 (tie-to-even)
leave the LSB to 0

stage-1 ⇒ Unpack, Compare & Align

- Compare Exponents: $d = E_A - E_B$
- Extract sign and append hidden bits to form the 24-bit significands
- Shift the smaller significant right by d bit using a barrel shifter to generate significand with G, R, S

stage-2 ⇒ significant Add/subtract & LZD

- Add/sub the 27-bit significant
- determine if there is a carry-out
- Run a LZD parallel block to calculate the required shift distance (shift) to normal the result

stage-3 ⇒ Normalization shifting

- Apply the calculated shifted to the raw sum using the normalization barrel shifter to bring the first non-zero bit to the leading implicit position
- Adjust the exponent by subtracting the shift amount or increasing it if there was an addition carry overflow

stage-4 ⇒ Rounding & Packing

- Read LSB of the normalized significand along with G, R & S
- Apply the rounding mode (required 24-bit increase)
- Strip the leading 1 pack sign, Exponent and mantissa into the final 32-bit register.

FPU multiplier

$$Value = (-1)^{S_A \oplus S_B} \times (1.M_A \times 1.M_B) \times 2^{E_A + E_B - Bias}$$

* Phase 1 → Unpacking
  - Extract sign, Exponent, mantissa
  - Append implicit 1 to form 24-bit significant

* Phase 2 → Sign/Exp
  - Sign = S_A ^ S_B
  - Exponent: E_A + E_B - 127 ↳ Bias

* Phase 3 → multiply
  - multiply significant 24-bit x 24bit = 48-bit product

* Phase 4 → Normalize
  - shift product to fix 1.xxx format
  - Adjust exponent based on shift

* Phase 5 → Rounding
  - Apply Round-to-Nearest even (using G, R, S) bits
  - Handle potential rounding overflow

* phase 6 → Pack/Exception
  - Assemble final 32-bit IEEE 754 float
  - flag underflow / overflow / NaN

Phase 1 Unpacking and significant Extraction

We Unpack the 32-bit input into sign, Exponent and Mantissa fields. To represent the true fractional significant, we append the implicit leading 1 to the MSB of the 23-bit mantissa.

$Sig_A = 1.xxx$ (24 bits)
$Sig_B = 1.xxx$ (24 bits)

Phase 2 Sign Calculation and Exponent Addition

Sign Bit = The output sign is a simple XOR of the input sign

$$S_y = S_A \oplus S_B$$

Exponent Addition = We add the exponent. However bcoz both exponents are biased by +127 adding them together adds the bias twice. we must subtract one bias

$$E_{temp} = E_A + E_B - 127$$

Phase 3 Significand multiplication

$$Product_{raw} = Sig_A \times Sig_B$$

format: Bcoz both inputs are normalized (1.xxxxx ≥ 1.0 and < 2.0) the minimum possible product is $1.0 \times 1.0 = 1.0$ (binary 01.00...) and the maximum is $< 2.0 \times 2.0 = 4.0$ (binary 11.11...).

$Product_{raw}$ represents the integer position (either 01 or 10)

Phase 4 Normalization

Since the integer portion of our 48bit product is either 01 or 10 we only have two possible scenario to normalize the binary point back to 1.xxx...

Case A (no shift) 01.xxx

The significand is already normalized

$E_{final} = E_{temp}$

Case B (Right shift by 1) 10.xxxx / 11.xxxx

Right shift the significand by 1 position

$E_{final} = E_{temp} + 1$

Phase 5 Rounding

Assuming our normalized product is aligned so that bit 46 is the leading 1:

- mantissa [45:23] 23bits
- Guard (G) Bit 22
- Round (R) Bit 21
- Sticky (S) The logical OR reduction of all lower bits [20:0]

Rounding execution: We apply the Round-to-Nearest-Even rule if we round up, we add 1 to the LSB of our 24-bit significand

Rounding overflow: If adding 1 causes a carry-out we shift sig by 1 and increment E_{final} by 1

Phase 6 Packing and Exception Handling

Overflow: if $E_{final} \geq 255$ the result overflow to $\pm\infty$

Underflow: if $E_{final} \leq 0$ the result is flaged as underflow

Asseble: merge $\{s_y, E_{final}[7:0], M_{final}[22:0]\}$ into the final 32-bit output register.

Fused multiply-Accumulate (FMA)

Result = A×B + C

- Normally split multiple-and-add operation performs two separate roundings, one after the multiplication and one after addition.

- FMA perform the multiplication to full intermediate precision (48 bit) align the addend C directly with this 48-bit product add them and perform only a single rounding step at the very end.

This approach delivers
High precision & low latency

Phase 1 multiply
24x24 multiplier

Phase 2 Alignment
Align product and operand C to a common scale

Phase 3 Add/sub
78-bit wide Three-Input Adder/subtractor

Phase 4 Normalize
Leading Zero Anticipator (LZA) & Normalization

Phase 5 Rounding
Rounding the raw result by using GRS bit

Phase 6 Packing
Pack the raw result into 32-bit output

Phase1 Unpacking & Significand multiplication

we extract the sign, exponent and mantissa of A, B and C

- we multiply the 24-bit significant of A and B to yield 48-bit raw product

$$P_{raw} = Sig_A \times Sig_B$$

- The intermediate exponent of this product is

$$E_{prod} = E_A + E_B - 127$$

Phase2 Dual-path Alignment

Bcuz we are adding the 48-bit product to the 24-bit significand C we must align their exponent.

$$d = E_{Prod} - E_C$$

To prevent losing precision, we align both components inside a wide 78-bit register space

if $E_{Prod} \geq E_C$ we shift C's significand to the right by d bits

If $E_C > E_{Prod}$ we shift the 48-bit product to the right by -d bits

Phase3 The wide Addition/sub

We add or subtract the aligned 48 bit product and the aligned C significand based on their relative sign. Bcuz of the alignment shift, this adder must be wide (typically 78 bit) to preserve the high-precision product bits, guard, round and sticky bits all in one pass

Phase 4 Unified Normalization (LZA)

The result of our wide addition/subtraction might have leading zeros (due to cancellation) or an overflow carry

- We use LZA operating in parallel with adder to instantly calculate how many position we need to shift the result to restore 1.xxxx
- The exponent is adjusted accordingly

Phase 5 single-step Rounding and Packing

Now that we have a single, high-precision normalized value we extract target 23 bit mantissa along with G, R, S bits. We apply Round to Nearest Even rule check for overflow exception and pack the final 32 bit float register

Floating-point division

Unlike multiplication (which is computed combinationally) division is inherently sequential

$$Value = (-1)^{S_A \oplus S_B} \times \left(\frac{1+M_A}{1+M_B}\right) \times 2^{(E_A - E_B + Bias)}$$

Phase 1: Unpack sign, Exp and significand

Phase 2: Calculate sign, subtract Exponent with Re-Biasing

Phase 3: Core division

Phase 4: Normalization

Phase 5: Rounding

Phase 6: Exception / Packing

Phase 1 Unpacking and Pre-Computation checks

We extract the sign, exponent and mantissa fields. In division this phase is critical bcoz division-by-zero is a primary hardware exception.

- Div-by-zero check: if B=0 and A≠0 the FPU by pass the calculation engine entirely sets the division-by zero exception flag, and output ±∞ (matching the XOR of their sign)
- NaN/indeterminate check: if A=0 and B=0 or if both are ∞, the result is flagged as invalid and outputs a NaN (Not-a-number)
- Hidden Bit Append: for normal numbers, we append the implicit leading 1. forming

$$Sig_A = \{1'b1, M_A\}$$

$$Sig_B = \{1'b1, M_B\}$$

Phase 2 Sign Calculation & Exponent subtraction
While the division engine works on the significands the sign and exponent are computed in parallel

$$S_Y = S_A \oplus S_B$$

$$E_{temp} = E_A - E_B + 127$$

Phase 3 The Core Division Engine
We must calculate

$$Q = \frac{sig_A}{sig_B}$$

Bcz both significands are normalized they fall strictly in the range $1.0 \leq sig < 2.0$
Thus our quotient Q will always fall in the range
$0.5 < Q < 2.0$

Phase 4 Normalization
Bcez our new quotient Q is bounded btw 0.5 and 1.0 we only have two possible alignment

Case 1 $(1.0 < Q < 2.0)$ Bit 24 of the quotient is 1

$$E_{final} = E_{temp}$$

The quotient is already normalized

Case 2 $(0.5 < Q < 1.0)$ Bit 24 is 0 and 23 is 1

$$E_{final} = E_{temp} - 1$$

We must shift the quotient left by 1bit to pull the leading into the implicit integer spot

Phase 5 Rounding the Result

Calculating S without infinite division: The FPU does a reverse-ch
it multiplies the truncated quotient back by divisor

$$Remainder = Sig_A - (Q_{truncated} \times Sig_B)$$

If Remainder $\neq 0$ then $S = 1$
Remainder $= 0$ then $S = 0$

Phase 6 Exception Handling and Packing
We assemble the finished product into the standard 32-bit
Containers and perform final checks

Overflow: if $E_{temp} > 255$

underflow: $E_{final} \leq 0$